Leadership Feedback & Growth Stories

Too Hands-On / Not Delegating Enough

The most impactful piece of difficult feedback I received came from my manager about a year into my EM role. The feedback was direct: I was too hands-on. I was reviewing too much code, attending too many technical discussions as a participant rather than a facilitator, and making decisions my engineers should have been making themselves.

My first reaction was defensive — internally, at least. I told myself I was doing it because I cared about quality. Because I knew the codebase. Because it was faster if I just handled it. All of those things were true. And all of them were the wrong reason to be doing what I was doing.

The Impact

What my manager helped me see was the second-order effect I was missing entirely. Every time I stepped in to review a decision or catch a problem myself, I was sending an implicit message to my engineers: *I don't fully trust you to handle this.* I wasn't saying it. But I was communicating it through every action. And over time that communication was doing real damage — engineers were waiting for my input before moving forward, bringing me problems they should have been solving, and not developing the judgment they needed because I kept short-circuiting the process.

What Changed

The feedback landed hard because I could see it was true the moment it was named. I had confused caring about outcomes with controlling the path to outcomes. Those are very different things.

What I changed was structural. I made a deliberate rule for myself: before stepping into any technical decision, I ask one question first — **is my involvement here developing this engineer or replacing their development?** If the answer is replacing, I stay out. I ask a question instead of giving an answer. I create space for them to work through it even when I can see the solution clearly.

The Proof

The clearest proof that it worked came during the period when I had competing priorities across multiple workstreams on the paid hints initiative. I had to distribute my attention across five teams and three parallel tracks simultaneously. Because I had already built the delegation muscle, I could hand outcomes — not tasks, outcomes — to my staff engineers and trust them completely. The experiment track ran almost entirely without me. That wouldn't have been possible if I hadn't received and genuinely acted on that feedback.

Key Takeaway

The irony is that stepping back made me more effective as a leader, not less. The team made better decisions when they owned them. Engineers grew faster when I wasn't in the way. And I had more capacity for the work only I could do — the cross-team alignment, the stakeholder management, the strategic decisions — because I'd stopped doing the work my engineers were better positioned to do.

What I took away is that the hardest part of becoming an EM isn't learning new skills. It's unlearning the instincts that made you a great IC. Hands-on was my superpower as an engineer. It was my blind spot as a manager — and I needed someone to name it before I could see it.

Follow-up: Are you still working on it?

Honestly, yes — it's not a problem I solved once and closed. It's a tendency I manage actively. The trigger for me is usually when a project is moving slower than I think it should, or when quality feels at risk. That's when the instinct to step in comes back strongest. What's changed is that I now recognize the trigger and ask myself the question before I act: is my involvement here developing this engineer or replacing their development? That pause is usually enough to redirect me toward the right response. But I'd be overstating it if I said the instinct is gone. It isn't. I've just built a better filter for it.

Moving Too Fast Without Bringing People Along

The feedback that changed how I operate most came from a situation I created myself — and didn't fully recognize until someone named it for me.

I build first and do a POC which will be better if I can first do a kickoff and then do a quick POC. When I was building the user journey logger application, it could have helped me identify some issues earlier as the New Relic team pointed out some tech issues. They suggested building it via nerdlet instead of building a standalone React app as it gives me 10X more API requests.

Now I setup a kickoff call even when I have just the initial concept.

Shielded the Team Too Much

The most uncomfortable upward feedback I received came during a retrospective after a particularly turbulent period at Ancestry — the period when our entire product and UX org was laid off overnight. I had made a deliberate decision during that time to absorb as much of the org uncertainty at my level as possible. My reasoning was clear in my own head: my team didn't need to be distracted by noise they couldn't control. My job was to keep them focused and productive while I navigated the ambiguity above.

What I Missed

What I didn't realize was how that decision landed on the other side.

During the retrospective, one of my most trusted senior engineers said something that stopped me cold. He said — and I'm paraphrasing — *"I appreciate that you were trying to protect us, but there were moments where I found out about things that affected our work*

from people outside the team before I heard them from you. That made me feel like I wasn't trusted with the full picture."

That was hard to hear. Not because it was unfair — it was completely fair. But because my intention had been the opposite of the impact. I thought I was protecting them. They experienced it as being kept in the dark.

The Key Distinction

The distinction I had missed was the difference between shielding engineers from noise they can't act on and withholding information that affects how they understand their own situation. Those are completely different things. Org politics they can't influence — shield them from that. Information about the direction of the team, the status of the org, decisions that affect their work — they have a right to that, even when it's uncertain and incomplete.

Structural Changes

What I changed was structural. I introduced a standing five-minute segment at the end of our weekly team meeting specifically for org context — what I knew, what I didn't know yet, and what I was doing to find out. I called it explicitly: here's what's happening above us, here's the uncertainty I'm sitting with, here's what I'll share as soon as I have it. That transparency didn't create anxiety — it actually reduced it. Because the team stopped filling the silence with speculation, which is always worse than incomplete information.

The second change was more personal. I started asking myself a specific question before deciding what to share: **am I withholding this because sharing it would genuinely harm the team, or because I haven't processed it myself yet and it's easier not to share?** The second reason is never good enough. If I'm sitting with uncertainty, my team can sit with it too — as long as they know I'm sitting with it alongside them.

The Impact

The engineer who gave me that feedback told me months later that the change had made a meaningful difference — not just in how informed he felt, but in how much he trusted me. That feedback completing a loop was more valuable to me than almost any metric I tracked that year.

What I learned is that protection and transparency aren't opposites — but they can feel that way when you're in the middle of a crisis and moving fast. The instinct to shield is a good instinct. The discipline is knowing exactly what you're shielding people from and why — and being honest with yourself when the answer is your own discomfort rather than their benefit.

Follow-up: How do you decide what to share when information is sensitive or not fully confirmed?

My default is to share earlier with less certainty rather than later with more certainty. I'd rather say "here's something I'm hearing that isn't confirmed yet — I wanted you to know I'm aware of it and I'll update you as soon as I know more" than have my team find out from someone else. The moment an engineer hears something significant from outside the team before they hear it from me, I've lost something that's very hard to rebuild. The exception is information I've been explicitly asked to keep confidential by leadership — in that case I say directly: there are things happening that I can't share yet, but I want you to know I'm not keeping you in the dark by choice. That honesty about the constraint is itself a form of transparency.

Onboarding Rebuild Launch

At Ancestry, I was the engineering manager leading a full rebuild of our user onboarding flow. Product set a hard launch deadline tied to RootsTech, a major public event in the genealogy world. When full scope and designs landed one month in, we had three months left and a lot to build.

The Challenge

I started by running a t-shirt sizing session with the team, covering every major task and adding buffer tickets for unknowns. The estimate came back at **five months of work against a three-month window**. I didn't wait for the deadline to surface the problem.

The Approach

I immediately facilitated a prioritization exercise with the product managers, sorting every feature into must-have, nice-to-have, and can-come-later buckets. We aligned on a phased rollout:

- MVP at RootsTech
- Follow-up release two months later
- Full delivery shortly after

That conversation was only possible because I had concrete numbers to anchor it.

Mid-Flight Crisis

Mid-development, the team discovered that hint generation rates weren't matching the legacy flow. Hints are a core part of the Ancestry experience, and hint generation rate was one of our primary success metrics. I escalated it immediately as a launch risk, called an SOS with stakeholders, and explained clearly what was broken and why it mattered.

I then paired directly with a senior engineer to dig into the root cause. We found that a fundamental change in how user information was being added was driving the discrepancy. I brought the findings back to product with a specific trade-off proposal. I suggested moving one MVP feature, roughly equivalent in effort to the fix, out to the later release, and pulling the hint generation fix into MVP scope. I gave product a clear, data-backed option rather than just a problem to react to. They agreed, I re-sequenced the team's work, and we stayed on track.

Results

The MVP launched on time for RootsTech as an A/B test at 50% traffic:

- **Hint generation rate:** +2% (reversing the regression)
- **Build-through rate:** +5%
- **T1 to T2 retention:** +3%
- **Time to complete onboarding:** Dropped from ~1 hour to ~40 minutes
- Strong positive feedback from users at RootsTech

The MLP shipped two months later, and the legacy flow was eventually fully deprecated.

Lesson Learned

Estimation is the most powerful tool I have when negotiating scope with product. After this project, I changed how I run estimation sessions. I now require at least two dedicated buffer tickets for any project spanning two or more months, and I push the team to surface and examine unknowns at the start rather than discovering them mid-flight.

Proactive Onboarding Prototype

I was leading an engineering team working on a long-standing onboarding application when the existing product team was let go and a new one came in with a different vision. The new team needed time to ramp up before they could finalize requirements. My team had bandwidth and was looking for work. The stakes were real: a delayed launch, a stalled team, and a missed opportunity to ship a better user experience.

Taking Initiative

I didn't wait. I set up one-on-ones with the new product managers to understand their early directional thinking, even before anything was finalized. Pretty quickly I identified that the core of their vision centered on a tree-building functionality, a complex, foundational piece that would take significant time to build.

I made the deliberate call to start building toward that functionality without finalized requirements, knowing some code might be throwaway. I created generic engineering tickets and a working document covering:

- What could be built
- Where new data would come from
- How the UI might look
- What API calls could be put in place

Building Buy-In

Before moving forward, I brought my manager into the conversation, explained the approach and the tradeoff, and got his full support. I was equally transparent with my engineering team, telling them directly that some of this work might get thrown away. I wanted them bought in, not blindsided.

I also communicated clearly to the product team that our prototype work would not constrain or influence their independent vision-building process. From there, I continuously synced with product as their thinking evolved and pivoted the implementation with each new signal I received. I engaged architects throughout to validate feasibility and identify where new data points could be sourced. I also proactively surfaced engineering-driven enhancement ideas to the product team based on what we were already building, ideas that later became quick experiments after launch.

Results

When the product team finalized requirements and design delivered final specs one month later, my team already had a working solution that was very close to the target. Only minor gaps and UI updates remained.

- **Total build time:** Dropped from an estimated 4 months to 3 months (25% reduction)
- **Conversion:** +10% increase
- **NPS:** +15% improvement
- Engineering enhancements surfaced during the ambiguous phase became fast post-launch experiments that delivered early wins

Behavioral Shift

The biggest shift in my behavior afterward was structural. I introduced regular early vision meetings with product as an ongoing practice, where they share directional thinking before requirements are complete and engineering can begin prototyping if bandwidth allows. **Waiting for complete requirements is often the wrong default. Enough directional signal is enough to start building meaningfully.**

User Journey Analytics Tool

At Ancestry, I was a software engineer on a product and engineering team where a frustrating question came up constantly. Stakeholders would ask "What happened to this user journey?" and nobody could answer it confidently. Our logging in New Relic only captured errors, was hard to navigate, and gave no coherent picture of what a user actually did during a session. The closest third-party solution was FullStory at a million dollars. That wasn't happening.

Owning the Problem

This problem wasn't assigned to me. I just decided to own it. I started by reaching out to the internal New Relic team to explore what was possible with our existing data. They suggested grouping requests by a request ID, but I evaluated that approach and determined it wasn't enough to reconstruct a full user journey. So I decided to build something myself.

Building the POC

I built a proof-of-concept that read logs from New Relic and generated custom dashboards showing:

- What events a user triggered
- What pages they viewed
- How they moved between pages
- How long they spent on each one

Before going further, I brought the POC back to the New Relic team to validate it against company policy and get explicit sign-off that no user data was at risk. I also met with the product team to understand what insights would actually be useful to them, including whether dwell-time data could support experiment design.

Scoping Decisions

I made a deliberate call to scope out video session replay entirely, keeping the project focused, shippable quickly, and free of security risk. I built custom funnels and cohort analysis into the platform, then integrated it with Statsig, Ancestry's experimentation platform, so user

journey data and A/B test results could live in a single dashboard.

Building Momentum

Throughout the build, I continuously demoed progress to my manager and VP to maintain buy-in and surface concerns early. I demoed the finished tool at an all-hands meeting, onboarded four teams as a pilot, then presented at Ancestry's monthly engineering demo, which drove adoption across the broader engineering org. I made the project open-source internally and handed it off to the New Relic team so they could build on top of it.

Results

The results were significant:

- **Spike tickets for bug investigation:** Dropped by 75%
- **Time to diagnose bugs:** Reduced to 1/4 of previous time
- Product team identified several new experiment ideas directly from dwell-time data
- Tool spread beyond my team and became an internal open-source platform with contributions from multiple teams
- Gave Ancestry a practical alternative to a million-dollar vendor product

Lesson Learned

The New Relic team corrected a few of my early technical approaches, which added rework I could have avoided. I learned that **involving the right technical stakeholders at the idea stage, not after I've started building, saves time and surfaces what those teams have already tried.** I now set up kickoff meetings at the idea stage on every project, and it consistently helps things ship faster.

Paid Hints Experiment

As a software engineering manager at Ancestry, I was asked to help drive a key business metric called Build-Through Rate. The product team believed putting new person hints behind a paywall could move that number, but there was no infrastructure to support it and five separate engineering teams each owned a different piece of the flow.

Understanding the Problem

Before touching any technical planning, I partnered with product to deeply understand what outcome we were actually trying to achieve. Then I independently audited the existing infrastructure to understand how hints were surfaced and where the data came from. I met with architects to map out what would need to be built, and I reached out to all five engineering managers to understand their ownership.

Building Alignment

I built flow diagrams to clarify dependencies and organized a cross-team alignment call to present the current state and proposed approach. When the estimated timeline came back at five months, I pushed back. That was too long to wait before knowing if users would even pay for hints.

The Lightweight Experiment

So I proposed a lightweight experiment instead. I scoped it to:

- A single UI stack
- Zero free hints offered
- Paywall shown on first attempt
- No ledger system required

I coordinated with the team owning that UI and launched the experiment using Adobe Target in **two weeks**. While it ran, I kept working in parallel with architects on the larger infrastructure design.

Accelerating Delivery

When the experiment came back positive, I used that data directly with engineering managers and product stakeholders to shift prioritization across all five teams. I assigned my team to own the ledger system implementation and took on a significant portion of that work myself. I also built a shared integration component on my team's side so other teams wouldn't have to duplicate the effort, which cut ledger testing and implementation work roughly in half. I set up recurring sync calls with all engineering managers and product stakeholders to surface blockers fast and keep everyone moving.

Results

- **Lightweight experiment:** Live in 2 weeks, validated user engagement with paid hints
- **Full implementation:** Wrapped up in under a month (ledger development ran in parallel)
- **Testing and bug fixes:** 2 more weeks
- **Build-Through Rate:** +20% increase
- Successfully shipped the first ledger system ever built at Ancestry
- Shared integration component cut implementation and testing effort for other teams by ~50%

Key Takeaway

When a timeline feels too long to wait before testing a hypothesis, don't just accept it. Look for the smallest possible experiment that can validate your assumptions first. Running that lightweight test in parallel with infrastructure planning let us de-risk the project and build momentum at the same time, and I now treat that as a default pattern.

PR Review Bottleneck Fix

I was an Engineering Manager leading a team of 13 developers building a full revamp of our onboarding application. We sat within the growth org, so our mandate was clear: ship fast, run experiments, and drive revenue. Early on, with a smaller team, delivery was smooth. But once the full team was contributing, things slowed down noticeably.

The Problem

A senior engineer — genuinely talented, deep React expertise — was leaving extensive PR comments, mostly around style: naming conventions, indentation. PRs were getting blocked, engineers were hesitant to even open them, and after reviewing two sprints, our output was well below expectations.

Diagnosing the Issue

As the Engineering Manager, it was on me to fix this. I started by calling a team-wide meeting to surface the issue, but quickly realized I wasn't going to get candid answers in a group setting — people were too guarded. So I shifted approach and used my existing 1:1s to gather honest, private feedback instead.

The pattern was consistent: PR reviews were getting stuck on style comments — naming conventions, indentation — rather than anything substantive.

The Difficult Conversation

Armed with that, I scheduled a direct, private conversation with the senior engineer. I kept everyone else anonymous to protect psychological safety, but I was clear about the business impact: his review style was slowing our ability to ship and run experiments, and it was demoralizing the team.

I also acknowledged his positive intent — I wanted to redirect his expertise toward mentorship, not shut him down. He pushed back initially, resistant to the idea that his reviews could be demoralizing. I stayed patient and kept explaining the impact until he became open to trying something different.

The Solution

Together, we co-created a PR comment tagging system:

- **Essential Fixes** — blocks merge
- **Suggestions** — nice to have
- **Nit Picks** — optional

This gave developers clarity on what actually blocked a merge.

I also introduced **Husky** to automate ESLint and indentation checks at commit, eliminating the most common style issues before they ever reached a PR. Finally, I established a dedicated 30-minute PR review session right after standup each day to keep reviews moving consistently.

Results

The results came quickly and were hard to miss:

- **Average PR merge time:** Dropped to within 24 hours
- **Sprint velocity:** Increased by 30%
- **Deployment frequency:** Pushing code to production once every three days
- Team dynamic genuinely changed — engineers stopped hesitating before opening PRs
- Senior engineer shifted into a mentorship role, coaching teammates on code quality rather than blocking merges

The Validation

The moment I knew it had truly landed was during a team retrospective, when another senior engineer spontaneously called out the improvements — specifically crediting the new PR process and linting setup for helping the team ship faster. That kind of organic recognition told me we hadn't just patched a problem. We'd built something the team actually valued.

It reinforced for me that resolving conflict well isn't about picking a side — it's about finding a structure that works for everyone.

Reviving Team Creativity

As an engineering manager leading a 12-person team, I noticed concerning signs during our four-month project to build a new onboarding flow. We were maintaining the existing system while building its replacement, and the pressure was showing.

The Warning Signs

- Team discussions became repetitive cycles of the same solutions with zero creative thinking
- Everyone was so focused on delivery that no one wanted to explore new approaches
- No time allocated for learning
- People were grinding through tasks without any joy in the work
- I realized I'd fallen into the same trap, having not learned anything new myself in months

Taking Action

As the engineering manager, I was responsible for maintaining team morale and preventing burnout. I needed to create dedicated time away from our relentless product work so the team could think creatively and explore the bigger picture.

First, I discussed this learning initiative with my manager, explaining how our team had lost its creative spark and needed dedicated time away from delivery pressure. She immediately supported the idea, which gave me the backing I needed.

Implementing "No Meetings Friday"

Next, I reached out to our product owners and UX designers to establish **"No Meetings Friday"** — I was clear that this wasn't about avoiding work, but creating space for growth that would ultimately benefit our projects. I sent calendar invites to all twelve team members, marking them as tentative on Fridays to prevent automatic meeting scheduling. I specifically asked product owners to avoid their usual hour-long grooming sessions on Fridays, though I kept the door open for quick syncs if truly urgent issues arose.

Leading by Example

Then I realized I needed to model this behavior myself. I started using Fridays to dive into AI technologies, particularly LangChain, and began building a chatbot project to understand how these tools actually work. Finally, I created a structure where team members could demo their Friday learnings — whether new programming languages, tools, or even just interesting ideas they'd explored. This gave everyone a platform to share knowledge and celebrate their growth with the team.

Results

The results exceeded my expectations:

- Team responded enthusiastically to the initiative with an immediate energy shift
- Instead of repetitive solution cycles, people started bringing genuinely creative approaches to our onboarding project
- Team began suggesting new functionality and feature ideas to product partners
- Several engineers dove into AI technologies and are now eager to implement these new skills in actual work
- Team is now collaboratively building "Ancestry University" on Fridays, providing user guidance across multiple flows

Broader Impact

The product team has taken notice of this concept and we're already planning to implement the Ancestry University approach in other areas of the app and broader Ancestry journey.

Most importantly, I learned that protecting time for growth isn't a luxury — it's essential for maintaining both team performance and morale. When people have space to learn and explore, they bring that creativity back to everything they do.

Time After Product Layoff

Last July, I was managing a 12-person engineering team at Ancestry working on our onboarding flow when our entire product and UX teams were suddenly laid off during a strategic pivot. After three years of tight collaboration with daily back-and-forth on requirements, we were left without product direction for a month. When a new product team was finally assigned, they needed another month just to understand our work and develop a plan.

The Crisis

Meanwhile, my team's morale plummeted:

- People stopped participating in meetings
- Engineers were updating resumes
- Constantly asking about job security
- Multiple projects were halted
- Running experiments had to be stopped since they no longer aligned with company strategy

Immediate Response

As the engineering manager, I knew I had to act quickly to prevent my team from completely falling apart. I immediately scheduled one-on-ones with all twelve team members to understand their concerns and gauge morale. During these conversations, I provided transparent communication about what I knew:

- Engineering wasn't being impacted
- No additional layoffs were planned
- Our technical skills were still relevant and not obsolete, unlike the product functions that were eliminated

Creating Direction

Rather than let the team sit idle, I stepped in to provide direction when we had bandwidth but no clear assignments. I proposed we use this transition period strategically to upgrade our skills, specifically focusing on AI capabilities that would benefit us long-term. I led the team in building AI-related projects, concentrating on technical improvements and learning how AI could enhance our day-to-day coding and overall engineering performance.

Simultaneously, I made the decision to continue working on our existing sprint backlog, operating under the assumption that the new product team would later evaluate what aligned with company priorities. This approach kept everyone productive while we waited for clearer direction.

Results

The results exceeded my expectations:

- **Complete team stability:** Not a single engineer left despite the uncertainty
- Team remained productive by continuing to work through existing backlog
- AI skills initiative positioned engineers ahead of industry trends
- Discovered practical applications for enhancing daily coding workflows
- New product team was impressed by our proactive approach and technical foundation

Transformation

Most importantly, I transformed what could have been a demoralizing period into a growth opportunity. My team emerged stronger, more skilled, and ready to collaborate effectively with new stakeholders.

The experience taught me that transparent communication and decisive action during uncertainty can actually strengthen team cohesion. By refusing to let my team drift aimlessly, I demonstrated that perseverance isn't just about pushing through — it's about finding opportunity within adversity and keeping people engaged when everything feels unstable.

Project Arborist

My team was tasked with building new packaging infrastructure featuring hard offer prices, hint reviews, and tree creation capabilities. The challenge was that this required integrating with six different open-source stacks we'd never worked with before, all owned by other teams with different priorities.

The Challenge

External contractors had shown this could quickly boost company revenue, so leadership wanted it completed in three months alongside our existing commitments. However, leadership wouldn't adjust other teams' priorities to support us, creating immediate tension when those teams pushed back against our pull requests.

Assessment and Planning

I started by sitting down with my team to conduct high-level estimations, carefully considering the dependencies between all six stacks. I then created extensive documentation detailing:

- Each involved stack
- Required infrastructure changes
- Effort estimates

This gave me concrete data to present a **six-month timeline estimate** to leadership, where I highlighted that we lacked ready designs and faced significant technical complexities they hadn't considered.

Building Collaboration

Next, I met individually with engineering managers from all five teams whose systems we needed to integrate with. I explained our requirements and assessed their capacity to support us. These one-on-ones were crucial for understanding each team's constraints and building relationships before bringing everyone together.

I then facilitated group meetings with all the engineering managers to map out dependencies and create an integrated roadmap. This helped everyone see the full picture and understand how their work connected to others. Finally, I coordinated with leadership to adjust priorities across teams and secure one month specifically for design work before development began.

Results

The results exceeded my expectations:

- Leadership understood our concerns and agreed to involve all five teams
- **Launched in 2.5 months** — only a half-month delay from our revised timeline
- One-month experiment generated meaningful data that validated our approach
- Infrastructure became a foundation that supported several future similar initiatives
- Engineering managers from all five teams developed much better understanding of each other's needs

Key Lessons

This experience taught me three critical lessons about conflict resolution: 1. **Open communication from the start** prevents most conflicts from escalating 2. **Realistic planning with stakeholder input** leads to better outcomes than optimistic timelines 3. **Early involvement of all affected parties** creates buy-in rather than resistance

---

Conflict with Architect

When we were planning the new onboarding application at Ancestry, I ran into significant pushback from our architect during a formal architecture review. His position was clear — build the new application on one of our existing stacks rather than creating something new.

The Disagreement

On the surface it was a reasonable position. Reusing existing infrastructure reduces risk, leverages known patterns, and avoids the overhead of building from scratch. But I had a fundamental disagreement with that approach — and I had data to back it.

My Concerns

There were two structural problems with building on the existing stack:

1. Deployment Dependency

We didn't own that stack — another team did. Every change we needed to push would require us to open a PR into their codebase and wait for their team to review and deploy it. For a new onboarding experience that needed to run frequent experiments and iterate quickly based on user behavior, this was a fundamental mismatch. We would have been permanently constrained by another team's bandwidth and release schedule.

2. Blast Radius and Coupling Risk

The existing stack served our core application — a product used by the vast majority of our user base. The new onboarding application was fundamentally different: designed for a much smaller, newer audience, with different traffic patterns, different experiment cadences, and different risk tolerance. Putting both on the same stack meant that any production issue in our new application could potentially impact the core product.

The Formal Proposal

I didn't walk into the architecture review with opinions. I walked in with a formal proposal covering four specific dimensions:

1. **Expected traffic analysis** for the new application versus the core product 2. **Experiment pipeline** we had planned — showing how frequently we anticipated needing to push changes 3. **Blast radius analysis** — showing impact of production errors if isolated versus coupled 4. **Tight coupling risk assessment** — showing specific scenarios where changes to the core stack could break the new application and vice versa

The Discussion

The architect pushed back hard on several of my assumptions:

- He felt the deployment dependency concern was overstated
- He felt the blast radius concern was manageable with proper feature flagging

I acknowledged both points but argued that:

- Negotiating faster release cadence creates ongoing coordination overhead
- Feature flagging adds complexity; architectural isolation was a cleaner solution

The Outcome

We didn't reach full agreement in that meeting. The architect was about 80% convinced — he accepted the deployment dependency argument clearly, was partially convinced on the blast radius concern, but remained skeptical that the overhead of a new stack was justified.

What I walked away with was approval to proceed — with the explicit understanding that we would run early experiments quickly and use the results to validate the architectural decision in practice.

Validation

The launch was successful and on time. The other team continued their work on the core stack without any interference from our deployments. The experiment cadence we ran in the first weeks after launch made it increasingly evident that the architectural separation had been the right call — we were pushing changes at a frequency that would have been impossible under the shared stack model.

The architect never became a full enthusiastic advocate for the decision. But he told me afterward that he was glad we'd had the discussion — that the formal proposal and the data had given him confidence the decision had been made rigorously rather than casually.

Key Lesson

When you're pushing back against someone with institutional knowledge and authority, the worst thing you can do is make it a battle of opinions. Opinions create defensiveness.

Data creates the conditions for genuine reconsideration.

The architect wasn't being unreasonable — he was applying a valid heuristic from years of experience. My job was to show him specifically why this situation was different from the pattern his heuristic was built on. That required precision, not persuasion.